

Computational Economics

Lab: Introduction to Git and GitHub

Version control, project history, and collaboration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

A Project Folder with a History

Example: Recover an Earlier Version

- ▶ You edit a working program, but the new version stops running.
- ▶ You want to compare the versions, find what changed, and recover the working code.
- ▶ **Version control** records selected versions of project files and their history.
- ▶ By the end of the exercise, you will have a local repository, two meaningful commits, and a linked GitHub repository.

Git and GitHub Have Different Roles

	Git	GitHub
What is it?	Version-control software	A service that hosts Git repositories
Where?	Your computer	An online account
What does it do?	Records and compares commits	Shares code and supports collaboration
When needed?	While working locally, including offline	When publishing or exchanging changes

A **repository** combines project files with Git's history and metadata. A **remote** is a configured connection to another repository, often on GitHub.

From an Edit to a Shared Version



- ▶ Saving a file updates the working copy.
- ▶ A **commit** records a snapshot of the staged content in local history, with an author and message.
- ▶ A **push** sends local commits to the remote. A local commit alone does not update GitHub.
- ▶ A **pull** fetches remote changes and integrates them into the current branch; conflicts may need attention.

One Small Change, One Clear Commit

Example: Improve the README

1. Open the project folder in VS Code and inspect its Source Control panel.
2. Add a sentence explaining how to run the program.
3. Inspect the **diff**: the comparison between old and new content.
4. Follow the new instructions to check that they work.
5. Stage the intended files and commit with a useful message, such as “Add instructions for running the program.”

Before committing, check that the diff contains only the changes you intended.

Choose What Belongs in the Repository

Track and share

Source code: `.py` and `.ipynb`
A README explaining how to run the work
A list of required packages
Small, relevant results when requested

Keep outside the shared history

Local environment: `.venv/`
Caches and notebook checkpoints
Passwords, access tokens, and private data
Large generated files that can be recreated

- ▶ A **.gitignore** file lists patterns for files Git should leave untracked.
- ▶ Ignoring an already tracked file does not remove it from the repository or its history.
- ▶ Check notebook outputs as well as code before pushing. A public repository makes its tracked content accessible to others.

Connect Your Folder to GitHub

1. Install Git, register for GitHub, and open your project folder in VS Code.
2. In Source Control, initialize the folder as a Git repository. Set your commit name and email when prompted.
3. Add a README and a `.gitignore`; inspect, stage, and commit them.
4. Choose **Publish to GitHub**, sign in through the browser, choose a repository name and visibility, and check which files are included.
5. Open the resulting GitHub URL and confirm that your commit and files appear.

This workflow creates the GitHub repository and connects the local folder. If you already have a GitHub repository, clone it first and work in that local folder.

Practice: Is the Change on GitHub Yet?

Example: A Missing README Update

You edit and save the README, make a local commit, and open GitHub. The website still shows the old README.

- ▶ Which steps have completed: save, stage, commit, push?
- ▶ What would you check before trying again?
- ▶ After pushing, where can you verify the commit message and changed file?

Repeat with a second small edit. Explain the difference between the local history and the remote history to a partner.

Homework: Create and Connect Your Repository

1. Install Git and confirm it is detected by VS Code or with `git --version`.
2. Register for GitHub and complete account verification.
3. Create a project folder named `computational-economics`; initialize Git inside it.
4. Add a README and `.gitignore`; make an initial commit.
5. Create and connect your GitHub repository, then push. Make a second meaningful change, commit it, and push again.

Completion evidence: repository URL, two commit messages/identifiers visible on GitHub, and a short README explaining the project and how to run its Python files.

You may use the graphical interface. Be able to explain save, stage, commit, and push in your own words.

References and Further Reading

1. **Prof. Feng Zhigang (2026), AI for Economic Research: Dynamic Models, Language, and Agents. [Online course]**
 - ▶ Session 1.5: The Workbench: Python, VS Code, Git and GitHub
2. **Microsoft, Visual Studio Code Documentation. [Webpages]**
 - ▶ Quickstart: use source control in VS Code
3. **GitHub, GitHub Docs. [Webpage]**
 - ▶ Creating an account on GitHub
4. **Scott Chacon and Ben Straub, Pro Git. 2nd ed. [Book]**
 - ▶ Installing Git